simple_tutor — Engineering Handover

AI Tutor platform · multi-turn tutoring engine and evaluation harness

Prepared 2026-07-21 · branch pixeldesignlabs-dev-portuguese · covers evaluation fix-cycles 0–11 (Jul 16–21, 2026)

1. Purpose and scope of this document

This document hands over the simple_tutor engine — the production tutoring engine of the AI Tutor platform — for contiuous development. It consolidates three living documents maintained in the repository: the engine handover report (evals/reports/engine_handover_2026-07-20.md), the bottleneck analysis that started the latest fix arc (evals/reports/multi_turn_bottlenecks_2026-07-18.md), and the per-cycle change log (offline_eval/multi_turn_results/fixcheck/_cycle_log.md). Everything stated here is reproducible from those files and from the run artifacts referenced throughout.

The short version: over twelve evaluation cycles in six days, the multi-turn pass rate of the engine across three evaluation models rose from 17/60 (28%) to 53/60 (88%), with every deadlock class and every wrong-verdict class eliminated. The engine did not get a bigger model or a longer prompt — it got a series of small, evidence-driven protocol repairs, each traced to specific failing transcripts, implemented test-first, and verified by re-running the identical twenty-scenario benchmark.

2. What the engine is

apps/tutoring/simple_tutor/ is the default production engine (the kill-switch SIMPLE_TUTOR_ENGINE=off reverts to the legacy conversational tutor). Its core design decision is that the platform — not the language model — owns question state. When the tutor LLM wants to ask a question it must register it through a pose_question tool call, which the engine persists as the session's single InFlightQuestion row ("the slot"). When the student answers, the LLM calls record_answer and a server-side grader compares the answer against the slot's stored reference. The model is never asked to remember which question is live, because an earlier architecture that trusted it to do so mis-graded whenever one turn mixed praise with a new question.

Each student message triggers at most two LLM calls. Call 1 lets the model decide which tools to fire (grade the answer, pose a question, request a figure). The engine dispatches those tools server-side, applying its guard stack (next section). Call 2 feeds the tool results back so the model composes the student-facing reply with the grading verdict in hand; Call 2 also carries the repair mechanism — when Call 1 skipped a tool the protocol expected, Call 2 is constrained to make exactly that call, so a repaired turn costs no more than a normal one.

Turn behaviour is derived, never stored: a slot plus an answer-like student intent means the turn is a grading turn; no slot means a teaching/posing turn; conversational intents (clarification, pushback, off-topic, non-engagement — classified deterministically) suppress grading and leave the slot live.

Per-model-family prompt variants exist for evaluation runs only (Qwen receives a Markdown variant; Gemini and Kimi receive targeted rule add-ons); production runs the base prompt with no family override.

3. The guard stack — and the incident behind each layer

Every defensive layer below exists because a specific failure was observed in an evaluation sweep and traced in transcripts. All are env-toggleable, and most are gated to non-Anthropic model families, meaning production behaviour is unchanged by them.

Layer	Why it exists (the incident)
Forced pose/grade + adaptive gate	Open-weight models narrate questions as prose without calling the tool (Qwen: 3% compliance unforced) → no gradable slot → session stalls. The gate keeps compliant models unforced until their first miss.
Call-2 repair	Providers that ignore tool_choice get the skipped tool re-demanded on Call 2 at no extra call cost.
Anti-desync guard (intent-gated)	Gemini posed over unanswered questions 161× in one sweep, swapping the question under the student. Blocks a new pose only when the student attempted an answer; "idk" still allows a pivot (the ungated version deadlocked — cycle 4's lesson).
Anti-repetition (reject once, then force-advance)	Re-asks of already-answered questions burned turns. First re-ask of an already-correct stem is rejected with corrective feedback; a re-pose is accepted (a turn must never end slotless) and the step force-advances after a streak.
Slot rendering + divergence strip	Cycle 7's headline fix: the model's visible question and its registered question could differ, so students were graded against questions they never saw. The question block is now rendered server-side from the slot; a divergent trailing prose question is stripped.
Semantic dispatch order	record_answer grades the question the student SAW: an existing slot is graded before a same-turn pose replaces it; a fresh pose never grades the current message; late registration of a previous prose question still poses first (cycle 8).
Pose-after-correct + auto-pose fallback	On correct verdicts models wrote the next question in prose only, or declined to pose at all ("That's it — well done." followed by nothing). Call 2 is required to register the next question, and if it still declines, the engine poses the next unused pool question deterministically (cycles 8–9).
Malformed-pose salvage	Strict validation initially REJECTED malformed poses (optionless MCQs, letter references on numeric slots). Models that cannot act on corrective feedback turned every rejection into a deadlock (kimi 5/20, qwen 4/20 in the first cycle-8 legs). Salvage converts them into the nearest valid slot: catalog options and correct letter adopted by stem match, or mcq ↔ short_numeric conversion. Engine guards for weak models must repair, not reject.
Catalog letter coherence	The prompt's MCQ letter-rotation discipline (an anti-B-bias rule) made models re-letter catalog questions

	whose option order is fixed — correct answers were then graded wrong for whole sessions. For catalog questions the correct TEXT is the authority and the letter is derived from the displayed options (cycle 9).
Vocabulary + tool-JSON scrub	Students saw "we're in POSE/TEACH mode" and orphan "[" bubbles from narrated tool JSON. A deterministic filter strips platform vocabulary, tool-JSON lines, and punctuation-only residue before persistence.
Slot-aware, phrase-rotating placeholder	The old fixed placeholder promised "Here's the next one:" with no question attached and repeated verbatim (a deadlock signature). It now renders the in-flight question and rotates its phrasing deterministically.
Pivot guidance at attempt ≥ 3	Sessions ground 15+ turns re-scaffolding one hard question against a disengaged student. The in-flight block now instructs a downshift: pose a strictly simpler question or move on.
Transient-error retry	Vertex 429/503 responses used to become fallback-reply deadlocks (kimi: 12/20 in one early cycle). Backoff retry absorbed 177 rate-limit hits in a single later sweep with zero deadlocks.

4. Evaluation infrastructure

The multi-turn harness (evals/runner.py) drives real engine sessions against an Anthropic-Haiku student simulator that plays five personas (capable, average, struggler, error-prone, non-responder). Every session is scored twice: deterministically (persona-aware turn budgets, repetition detection, tool-syntax leaks, expected end-state) and by a Sonnet-4.6 session-level rubric aligned to science-of-learning principles. The dataset holds 400 balanced scenarios; evals/lint_dataset.py asserts structure, balance, and — since cycle 7 — fixture content quality (float-noise stems, lost MCQ option texts, impossible probability premises). Run headers record git_sha, engine, and tutor_model.

The fix-cycle benchmark is a fixed 20-scenario seeded draw (--multi-turn --sample 20 --seed 5) run against three models via offline_eval/run_cloud.sh: gemini-2.5-flash (Google API), kimi-k2-thinking and qwen3-next-80b-instruct (Vertex AI Model Garden). The judge and student simulator stay constant across cycles, so cycle-over-cycle deltas isolate engine changes. Cycle-to-cycle noise is about ±2 scenarios per model (tutor sampling is non-deterministic; the judge runs at temperature 0). Data integrity rule: a scenario marked errored is infrastructure (judge or simulator overload), never model failure — re-run those before reading a board.

5. Results — all twelve cycles (cycles as columns)

Scenarios passed out of 20 per model, with the pass rate underneath. Cells marked * carry the model's previous result forward: that model was not re-run in that cycle because the cycle's changes did not touch its code paths (gemini after C8; kimi after C9). glm-4.7 and deepseek-v3.1 were evaluated in cycles 0–2 only and then retired from the sweep as already at ceiling — the fix arc targeted the three struggling models; a dash (–) marks cycles where a model was not evaluated. The total row is the 3-model tracked board (gemini + kimi + qwen), which is what every cycle-over-cycle delta in this document refers to.

Model	C0 Jul 16	C1 Jul 17	C2 Jul 17	C3 Jul 17	C4 Jul 18	C5 Jul 18	C6 Jul 18	C7 Jul 20	C8 Jul 21	C9 Jul 21	C10 Jul 21	C11 Jul 21
gemini-2.5-flash	7/20 35%	5/20 25%	4/20 20%	8/20 40%	5/20 25%	9/20 45%	10/20 50%	17/20 85%	18/20 90%	18/20* 90%	18/20* 90%	18/20* 90%
kimi-k2-thinking	3/20 15%	7/20 35%	10/20 50%	12/20 60%	9/20 45%	11/20 55%	9/20 45%	13/20 65%	14/20 70%	18/20 90%	18/20* 90%	18/20* 90%
qwen3-next-80b	7/20 35%	8/20 40%	11/20 55%	9/20 45%	8/20 40%	8/20 40%	7/20 35%	11/20 55%	15/20 75%	14/20 70%	16/20 80%	17/20 85%
glm-4.7 (retired C2, at ceiling)	18/20 90%	15/20 75%	15/20 75%	–	–	–	–	–	–	–	–	–
deepseek-v3.1 (retired C2, at ceiling)	15/20 75%	15/20 75%	16/20 80%	–	–	–	–	–	–	–	–	–
3-model total (of 60)	17/60 28%	20/60 33%	25/60 42%	29/60 48%	22/60 37%	28/60 47%	26/60 43%	41/60 68%	47/60 78%	50/60 83%	52/60 87%	53/60 88%

What shipped in each cycle:

Cycle	Change shipped	Outcome / lesson
C0	Baseline	17/60. Deadlocks and timeouts dominate.
C1	Turn budgets raised (step+8); transient-error retry in the LLM clients	Deadlocks 15→0, but pass rate flat: rate-limit deadlocks became timeouts.
C2	Persona-aware turn budgets (struggling ×3, average ×2); judge/simulator retry	+5. Remaining failures become model-specific.
C3	Anti-desync guard: block posing over an unanswered question	+4 (gemini orphaned poses 161→23), but the ungated guard created new "idk" deadlocks.
C4	Intent-gated anti-desync (pivot allowed on non-engagement)	Dip to 22 — gemini flakiness; the gate itself was right and stayed.
C5	Per-family prompt variants (kimi lean/thinking, gemini anti-spiral, qwen brevity)	+6. Targeted rubric items moved; error-localization stayed flat.
C6	Anti-repetition guard; MCQ-fabrication fix (short_numeric	26/60. The 2026-07-18 bottleneck analysis of this cycle found the

	enabled)	prose/slot question desync as the dominant failure.
C7	Slot rendering + divergence strip; pose-after-correct; vocabulary scrub; repeat-reject; slot-aware placeholder; fixture repairs (float noise, 12 re-authored MCQs)	+15 — the largest single-cycle gain. "Logically consistent" rubric failures 26→9.
C8 / C8b	Semantic dispatch order; MCQ value→letter grading; pose validation → SALVAGE; pivot guidance; tool-JSON scrub	First legs collapsed for kimi/qwen (~90 pose rejections each became deadlocks) — the salvage re-legs recovered to 47/60. Lesson: guards for weak models must repair, not reject.
C9	Catalog letter coherence (correct text is the authority); rotation rule scoped to authored questions; auto-pose fallback	kimi 14→18 with zero logical-consistency failures; 50/60.
C10	Qwen prompt-variant iteration (spent micro-steps, precision acceptance, answered-question-finished, hint-without-reveal example, number sanity, opener variety)	qwen 14→16, average session 13.4→10.5 turns; 52/60.
C11	Fixture probability-template fix (5 → 5/9) + two lint rules; dataset consistency: 9 pre-cycle-2 budgets aligned to the steps×3 formula, 12 censoring sim-caps raised	qwen 16→17; 53/60. Honest decomposition: under the old budgets the score would be 16 — but the content fix removed the last deadlock and two scenarios now pass at or below their OLD budgets.

6. The diagnosis method (why this worked)

Every cycle followed the same loop: read the failing transcripts (not just the scores), name the mechanical failure class, fix it at the most deterministic layer available (engine code over prompt text over model choice), write the failing test first, smoke-test the exact scenario that exposed the failure, then re-run the identical benchmark. Three findings from this arc generalise:

- Identical failure signatures across unrelated model families point at the engine protocol, not the models. The cycle-6 analysis found the same two rubric items dominating gemini, kimi, and qwen — the fix (server-side question rendering) lifted all three at once.
- Guards must salvage, not reject, when the counterparty cannot self-correct. Strict pose validation was correct in principle and catastrophic in practice for models that ignore corrective errors; converting malformed poses into the nearest valid slot recovered them fully.
- When sessions become pedagogically clean, remaining failures migrate into the measuring stick — stale turn budgets and broken catalog content. Content lint rules and budget-formula consistency are as much a part of engine quality as the engine code.

7. Open items for the incoming owner, in priority order

- Ratio-1.5 budget band: 10-step struggling-persona scenarios with 15-turn budgets (e.g. engagement_recovery_non_responder_1144_07, now rubric-perfect at 23 turns, still failing on budget). Deciding whether this band moves to the documented steps×3 formula is a policy call — it flips scores, so it was deliberately left for the owner.
- straight_line_average_1145_16: the one genuine rubric-quality residue (4 low items) — next prompt/judge iteration target.
- Production content backfill: the float-noise and dropped-denominator template bugs are fixed at the generator, but production DB content generated before the fixes may carry the same artifacts the eval fixtures did.
- The flat rubric ceiling (error-localization ~0.49, mistake-recognition ~0.51) has not moved through prompt tuning; the grader's justification is now surfaced in the Call-2 verdict block — feeding it into hint text is the obvious next lever.
- Specialist judges under apps/tutoring/judges/ are deprecated pending unified-judge parity data; do not build on them.
- Last-step gap: pose-after-correct is skipped on the lesson's final step (to avoid colliding with the exit-ticket handoff), so prose/slot desync remains possible there. Bounded, but real.

8. Runbook

Engine unit tests (491, all green at handover):

```
./venv/bin/python manage.py test apps.tutoring.simple_tutor
```

Dataset + fixture lint:

```
./venv/bin/pytest evals/test_dataset_balance.py
```

One scenario against one model (fixtures must be loaded first):

./venv/bin/python manage.py loaddata evals/fixtures/institution.json evals/fixtures/lessons.json

GOOGLE_CLOUD_LOCATION=global TUTOR_MODEL_OVERRIDE="vertex_model_garden/qwen/qwen3-next-80b-a3b-instruct-maas" ./venv/bin/python manage.py run_eval --scenario <scenario_id>

The 3-model fixcheck sweep (~4–5 h):

RESULTS_DIR="$PWD/offline_eval/multi_turn_results/fixcheck/cycle12_<name>" CLOUD_MODELS_FILE="$PWD/offline_eval/cloud_models_3.txt" MODE="--multi-turn --sample 20 --seed 5" bash offline_eval/run_cloud.sh

Vertex authentication uses the gitignored isolated gcloud configuration (.env carries CLOUDSDK_CONFIG). Sweep cost: gemini and qwen ≈ 45 minutes each, kimi ≈ 2.5–4 hours (thinking model plus rate-limit backoff).

9. Source documents

- evals/reports/engine_handover_2026-07-20.md — full written handover (this document's primary source).
- evals/reports/multi_turn_bottlenecks_2026-07-18.md — the cycle-6 bottleneck analysis that opened the fix arc.
- offline_eval/multi_turn_results/fixcheck/_cycle_log.md — per-cycle change log, cycles 1–11.
- offline_eval/multi_turn_results/fixcheck/<cycle dirs> — raw run JSONs and logs for every cycle.
- Interactive briefing (charts, private link): https://claude.ai/code/artifact/a6ffba6c-d092-4634-8910-c0c1f58dcf27